MERL
Accelerating Engineering Innovation

MERL Training Sessions

GENIFY
LECTURE 01
16-July-2025

TOPICS COVERED:

- GENIFY OVERVIEW
- PYTHON BASICS
- DIGITAL LOGIC DESIGN

Presenter

Shahzaib Kashif

Co-Presenter

Shayan Hassan Baig

About the Instructors

Shahzaib Kashif

Software Engineer

Shayan Hassan Baig

Computer Scientist

GENIFY Course Content

A.I Concepts

Machine Learning Models, Deep Learning Models and Neural Networks

Linux, Git & Github

Essential tools for modern development workflows and collaborative projects.

Open Source Generative A.I

Explore open-source generative AI tools and frameworks relevant to hardware design.

RAG Implementation

Learn Retrieval-Augmented Generation techniques for AI systems in hardware contexts.

LLM Fine-Tuning

Techniques to fine-tune large language models for specialized hardware applications.

Chip Design

Fundamentals of modern chip design with a focus on AI-assisted methodologies.

Course Breakdown

	MODULE 01	MODULE 02	FINAL PROJECT
Artificial Intelligence	• Python Basics • Machine Learning • Deep Learning • Neural Networks	GenAI Specifics	GenAI + Chip Design Project
Chip Design	• Digital Logic Design • RISC-V Instruction Set Architecture • System Verilog HDL	RISC-V Single Cycle Core	

GenAI Roadmap

01

GENERATIVE A.I KEY CONCEPTS

- **Transformers**
  Backbone of GenAI models like GPT, BERT, T5 and more.
- **GANs**
  Generative Adversarial Networks (GANs) are designed to create new data such as images or videos.
- **Autoencoders**
  Unsupervised learning models used for data de-noising and data compression.
- **Variantional Autoencoders (VAE)**
  Adds a probabilistic approach to the encoding process
- **Attention Mechanism**
  Helps model focus on the important parts of the input sequence.

GenAI Roadmap

01
GENERATIVE A.I KEY CONCEPTS

Transformers
Backbone of GenAI models like GPT, BERT, T5 and more.

GANs
Generative Adversarial Networks (GANs) are designed to create new data such as images or videos.

Autoencoders
Unsupervised learning models used for data de-noising and data compression.

Variantional Autoencoders (VAE)
Adds a probabilistic approach to the encoding process

Attention Mechanism
Helps model focus on the important parts of the input sequence.

02
LANGCHAIN IN GENERATIVE A.I

Chatbot Webapp w. Langchain
Proof of concept project for real world use case implementation

Integrating LLMs w. Langchain
Langchain simplifies the integration of LLMs into real-world applications.

GenAI Roadmap

01 GENERATIVE A.I KEY CONCEPTS

- **Transformers**
  Backbone of GenAI models like GPT, BERT, T5 and more.
- **GANs**
  Generative Adversarial Networks (GANs) are designed to create new data such as images or videos.
- **Autoencoders**
  Unsupervised learning models used for data de-noising and data compression.
- **Variantional Autoencoders (VAE)**
  Adds a probabilistic approach to the encoding process
- **Attention Mechanism**
  Helps model focus on the important parts of the input sequence.

02 LANGCHAIN IN GENERATIVE A.I

- **Chatbot Webapp w. Langchain**
  Proof of concept project for real world use case implementation
- **Integrating LLMs w. Langchain**
  Langchain simplifies the integration of LLMs into real-world applications.

03 VECTOR STORES AND EMBEDDINGS

- **Vector Databases**
  To store vectors that are numerical representations of data for efficient search
- **Text Embeddings**
  Numerical vectors representing semantic meaning of text
- **Build a Semantic Engine**
  Search Engine to retrieve docs based on context and meaning

GenAI Roadmap

01
GENERATIVE A.I KEY CONCEPTS

Transformers
Backbone of GenAI models like GPT, BERT, T5 and more.

GANs
Generative Adversarial Networks (GANs) are designed to create new data such as images or videos.

Autoencoders
Unsupervised learning models used for data de-noising and data compression.

Variantional Autoencoders (VAE)
Adds a probabilistic approach to the encoding process

Attention Mechanism
Helps model focus on the important parts of the input sequence.

02
LANGCHAIN IN GENERATIVE A.I

Chatbot Webapp w. Langchain
Proof of concept project for real world use case implementation

Integrating LLMs w. Langchain
Langchain simplifies the integration of LLMs into real-world applications.

03
VECTOR STORES AND EMBEDDINGS

Vector Databases
To store vectors that are numerical representations of data for efficient search

Text Embeddings
Numerical vectors representing semantic meaning of text

Build a Semantic Engine
Search Engine to retrieve docs based on context and meaning

04
FINE TUNING OF LLM

GenAI Roadmap

01
GENERATIVE A.I KEY CONCEPTS

Transformers
Backbone of GenAI models like GPT, BERT, T5 and more.

GANs
Generative Adversarial Networks (GANs) are designed to create new data such as images or videos.

Autoencoders
Unsupervised learning models used for data de-noising and data compression.

Variantional Autoencoders (VAE)
Adds a probabilistic approach to the encoding process

Attention Mechanism
Helps model focus on the important parts of the input sequence.

02
LANGCHAIN IN GENERATIVE A.I

Chatbot Webapp w. Langchain
Proof of concept project for real world use case implementation

Integrating LLMs w. Langchain
Langchain simplifies the integration of LLMs into real-world applications.

03
VECTOR STORES AND EMBEDDINGS

Vector Databases
To store vectors that are numerical representations of data for efficient search

Text Embeddings
Numerical vectors representing semantic meaning of text

Build a Semantic Engine
Search Engine to retrieve docs based on context and meaning

04
FINE TUNING OF LLM

05
PROMPT ENGINEERING

GenAI Roadmap

01 GENERATIVE A.I KEY CONCEPTS

- **Transformers**
  Backbone of GenAI models like GPT, BERT, T5 and more.
- **GANs**
  Generative Adversarial Networks (GANs) are designed to create new data such as images or videos.
- **Autoencoders**
  Unsupervised learning models used for data de-noising and data compression.
- **Variantional Autoencoders (VAE)**
  Adds a probabilistic approach to the encoding process
- **Attention Mechanism**
  Helps model focus on the important parts of the input sequence.

02 LANGCHAIN IN GENERATIVE A.I

- **Integrating LLMs w. Langchain**
  Langchain simplifies the integration of LLMs into real-world applications.
- **Chatbot Webapp w. Langchain**
  Proof of concept project for real world use case implementation

03 VECTOR STORES AND EMBEDDINGS

- **Vector Databases**
  To store vectors that are numerical representations of data for efficient search
- **Text Embeddings**
  Numerical vectors representing semantic meaning of text
- **Build a Semantic Engine**
  Search Engine to retrieve docs based on context and meaning

04 FINE TUNING OF LLM

05 PROMPT ENGINEERING

06 A.I AGENTS

Chip Design Roadmap

01

CHIP DESIGN BASICS

- **Digital Logic Design**
  Learn about Gates and basic circuits using Logisim circuit simulator
- **RISC-V ISA**
  Learn about RISC-V Instruction Set Architecture (ISA) and its assembly language.
- **Verilog HDL**
  Learn the industry standard Hardware Development Languge (HDL), Verilog.

Chip Design Roadmap

01
CHIP DESIGN BASICS

Digital Logic Design
Learn about Gates and basic circuits using Logisim circuit simulator

RISC-V ISA
Learn about RISC-V Instruction Set Architecture (ISA) and its assembly language.

Verilog HDL
Learn the industry standard Hardware Development Languge (HDL), Verilog.

02
SINGLE CYCLE CORE

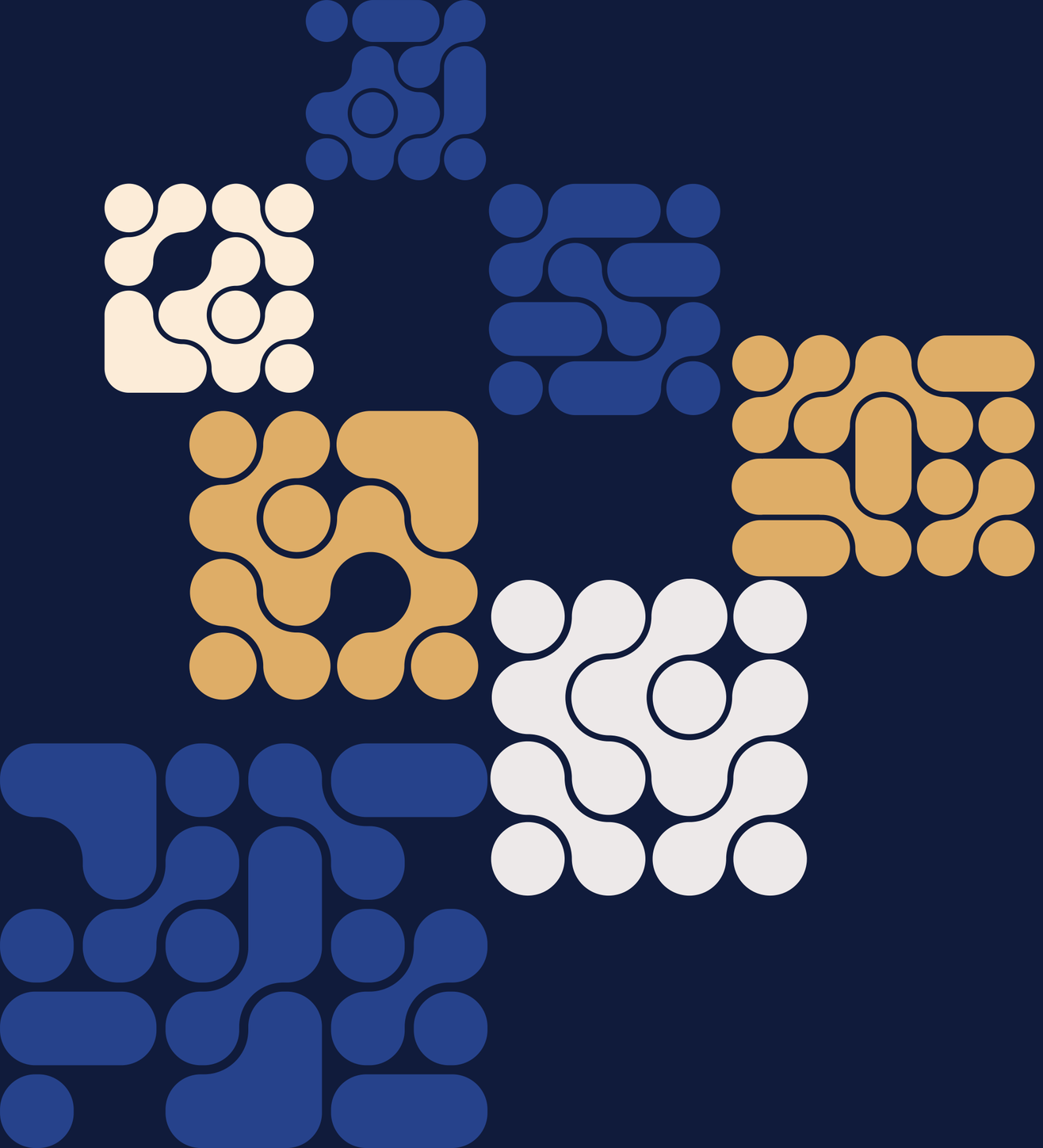
Logisim Implementation
Implement the RISC-V based Single Cycle Core in Logisim using Gates

HDL Implementation
Implement the RISC-V based Single Cycle Core in Verilog HDL

Core Verification
Verify the Core implementation using the standarized Compliance tests

BEFORE WE MOVE FORWARD, LET'S HAVE SOME DISCUSSION

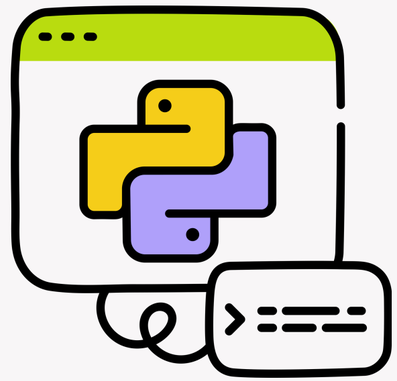
HOW MANY HAVE FAMILIARITY WITH PYTHON?

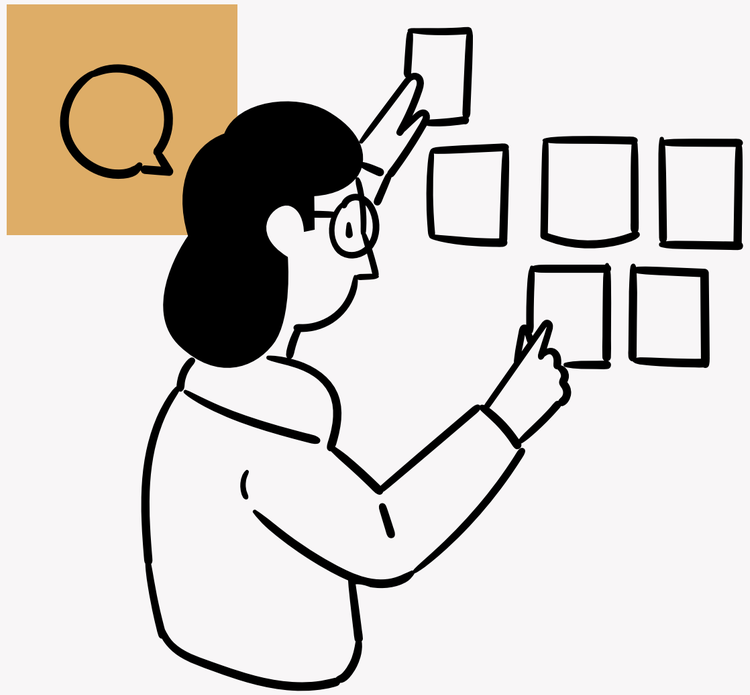


PYTHON BASICS

What is Python?

- High-level, interpreted programming language.
- Known for simplicity and readability
- Used in web development, data science, automation, and more





Why Python?

Easy to learn, versatile, and has a large community.

Python IDE

What’s an IDE?

An IDE (Integrated Development Environment) is a software application that provides tools like a code editor, debugger, and build automation [all in one place] to help developers write and test code efficiently.

Top Python IDEs and Enviroments

Jupyter Notebook
Web-based, cell execution, visualization support

Google Colab
Jupyter on cloud, free GPU, easy sharing

PyCharm
Full-featured IDE with smart code assistance

VS Code
Lightweight, customizable with Python extensions

Variables & Data Types

- No need to declare variable types.
- Common data types: int, float, str, bool

```
# Variable assignment
name = "Alice"    # String
age = 20          # Integer
height = 5.5      # Float
is_student = True # Boolean
print(name , age , height , is_student)


===== Output =====
Alice  20 5.5 True
```

Basic Operations

- Arithmetic: +, -, *, /, //, %, **
- Comparison: ==, !=, >, <, >=, <=

```
x = 10
2 y = 3
3 print ( x + y )   # Addition : 13
4 print ( x // y )  # Integer division : 3
5 print ( x ** y )  # Exponentiation : 1000
6 print ( x > y )   # Comparison : True
```

Conditional Statements

- Use if, elif, else for decision-making

```
age = 18
if age >= 18:
    print("Adult")
elif age >= 13:
    print("Teen")
else :
    print("Child")

==== Output ====
Adult
```

Loops

- for loop: Iterates over a sequence
- while loop: Runs until condition is false

```
# For loop
for i in range(5):
    print(i, end="␣") #Output : 0 1 2 3 4

# While loop
count = 0
while count < 3:
    print(count)
    count += 1
```

Data Structures

- Ordered, mutable collection
- Access elements by index (starts at 0)

```
fruits = ["apple" , "banana" , "cherry" ]
print (fruits [1]) # Output : banana
fruits.append("orange")
print (fruits) # Output : ['apple', 'banana', 'cherry', 'orange']
```

Dictionaries

- Key-value pairs, unordered, mutable

```
student = {"name" : "Alice", "age" : 20, "major" : "CS"}
print(student["name"]) # Output : Alice
student["grade"] = "A"
print(student) # Output : {'name': 'Alice', 'age': 20 , 'major': 'CS', 'grade': 'A '
```

Defining Functions

- Use def to create reusable code blocks
- Parameters and return values are optional

```
def greet(name):
    return f"Hello, ␣{name}!"

print(greet("Alice")) # Output : Hello , Alice !
```

Thank you for your attention!

Open floor for discussion.

ASSIGNMENT

WEEK-01

Python Excercises

- Python If-Else
  Easy, Python (Basic), Max Score: 10, Success Rate: 89.64%
- Arithmetic Operators
  Easy, Python (Basic), Max Score: 10, Success Rate: 97.28%
- Python: Division
  Easy, Python (Basic), Max Score: 10, Success Rate: 98.68%
- Loops
  Easy, Python (Basic), Max Score: 10, Success Rate: 98.06%
- Write a function
  Medium, Python (Basic), Max Score: 10, Success Rate: 90.36%
- Print Function
  Easy, Python (Basic), Max Score: 20, Success Rate: 97.42%
- List Comprehensions
  Easy, Python (Basic), Max Score: 10, Success Rate: 97.57%
- Find the Runner-Up Score!
  Easy, Python (Basic), Max Score: 10, Success Rate: 94.45%
- Nested Lists
  Easy, Python (Basic), Max Score: 10, Success Rate: 92.21%
- Finding the percentage
  Easy, Python (Basic), Max Score: 10, Success Rate: 97.36%
- Lists
  Easy, Python (Basic), Max Score: 10, Success Rate: 90.58%
- Tuples
  Easy, Python (Basic), Max Score: 10, Success Rate: 94.71%
- sWAP cASE
  Easy, Python (Basic), Max Score: 10, Success Rate: 98.22%
- String Split and Join
  Easy, Python (Basic), Max Score: 10, Success Rate: 98.61%
- What's Your Name?
  Easy, Python (Basic), Max Score: 10, Success Rate: 97.14%
- Mutations
  Easy, Python (Basic), Max Score: 10, Success Rate: 98.33%
- Find a string
  Easy, Python (Basic), Max Score: 10, Success Rate: 94.18%
- String Validators
  Easy, Python (Basic), Max Score: 10, Success Rate: 94.47%
- Text Alignment
  Easy, Python (Basic), Max Score: 10, Success Rate: 96.23%
- Text Wrap
  Easy, Python (Basic), Max Score: 10, Success Rate: 98.67%
- Designer Door Mat
  Easy, Python (Basic), Max Score: 10, Success Rate: 98.09%
- String Formatting
  Easy, Python (Basic), Max Score: 10, Success Rate: 92.59%
- Alphabet Rangoli
  Easy, Python (Basic), Max Score: 20, Success Rate: 96.19%
- Capitalize!
  Easy, Python (Basic), Max Score: 20, Success Rate: 83.26%
- The Minion Game
  Medium, Python (Basic), Max Score: 40, Success Rate: 87.28%
- Merge the Tools!
  Medium, Problem Solving (Basic), Max Score: 40, Success Rate: 93.99%
- itertools.product()
  Easy, Python (Basic), Max Score: 10, Success Rate: 98.10%
- collections.Counter()
  Easy, Problem Solving (Basic), Max Score: 10, Success Rate: 97.07%
- itertools.permutations()
  Easy, Python (Basic), Max Score: 10, Success Rate: 97.77%
- Polar Coordinates
  Easy, Python (Basic), Max Score: 10, Success Rate: 96.77%
- Introduction to Sets
  Easy, Python (Basic), Max Score: 10, Success Rate: 98.34%
- DefaultDict Tutorial
  Easy, Python (Basic), Max Score: 20, Success Rate: 93.14%
- Calendar Module
  Easy, Python (Basic), Max Score: 10, Success Rate: 97.03%
- Exceptions
  Easy, Python (Basic), Max Score: 10, Success Rate: 96.07%
- Collections.namedtuple()
  Easy, Python (Basic), Max Score: 20, Success Rate: 98.23%
- Time Delta
  Medium, Python (Basic), Max Score: 30, Success Rate: 91.70%
- Find Angle MBC
  Medium, Python (Basic), Max Score: 10, Success Rate: 89.39%
- No Idea!
  Medium, Python (Basic), Max Score: 50, Success Rate: 89.29%
- Collections.OrderedDict()
  Easy, Python (Basic), Max Score: 20, Success Rate: 98.16%
- Symmetric Difference
  Easy, Python (Basic), Max Score: 10, Success Rate: 98.05%
- itertools.combinations()
  Easy, Python (Basic), Max Score: 10, Success Rate: 97.00%

Hackerrank.com

Python OOP Excercises

Exercises link

RISC-V Excercises

1. Multiplication Table Generator

Question:

Write a RISC-V assembly program that generates and prints the multiplication table of a given number (e.g., 7) from 1 to 10. Store the result of each multiplication in a register.

Solution

2. Fibonacci Sequence in Two Registers

Question:

Write a RISC-V assembly program that calculates the Fibonacci sequence. Use 2 registers to Store initial Fibonacci sequence values (i.e 0 and 1) and then run the sequence until a given number N (e.g 6)

Example:

If N = 6, then at the end of program registers should contain 55 and 89 (11th and 12th Fibonacci numbers, 6 iterations done).

Solution